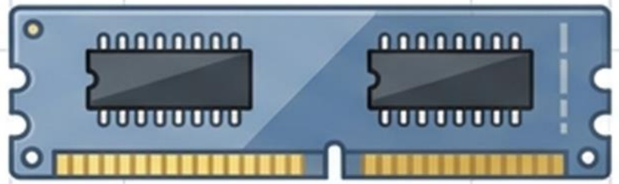
Instituto Profesional
Virginio Gómez
Universidad de Concepción

Integración de Aplicaciones con Bases de Datos

Persistencia, ADO.NET y Arquitectura DAO

ASIGNATURA: CONSTRUCCIÓN DE SOFTWARE (2026) | SEMANA 11
SEDE CONCEPCIÓN | UNIDAD III

De la Memoria Volátil a la Persistencia



Estado Actual: Memoria Volátil

Variables e instancias que desaparecen permanentemente al cerrar el programa (Ej: List<Pedido>).



Estado Deseado: Persistencia

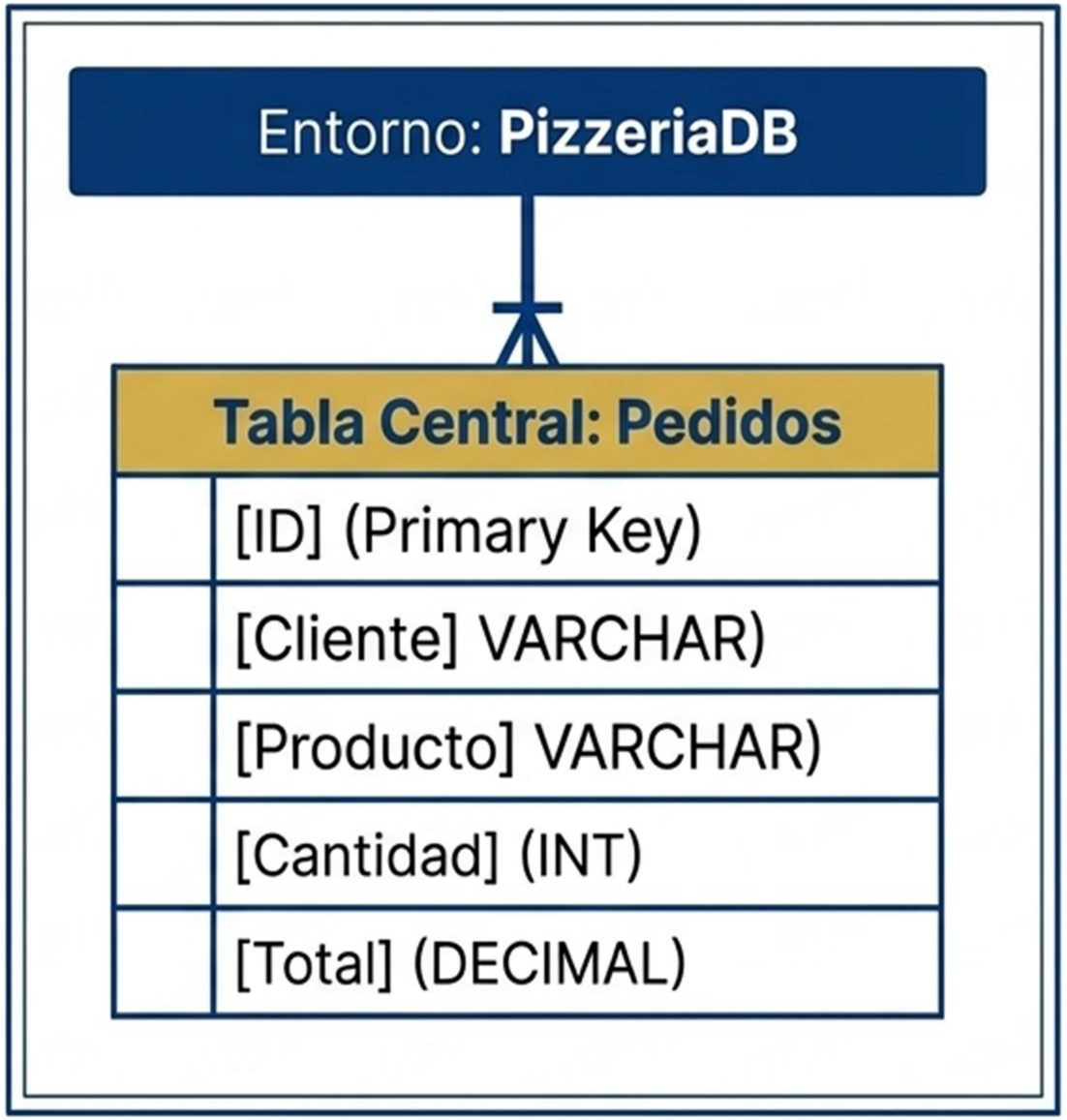
Datos estructurados que sobreviven de forma segura en el disco duro y pueden ser consultados a futuro.

Objetivo Semanal: Conectar nuestra aplicación C# a un motor de base de datos relacional para gestionar información de forma segura y estructurada.

Caso Práctico: Pizzería Bella Italia



El Problema: Necesidad imperativa de registrar, visualizar y gestionar las órdenes diarias de los clientes mediante una interfaz gráfica (WinForms), asegurando que la información comercial no se pierda al apagar el sistema.



Sesión 1: SQL Server Express (.\SQLEXPRESS)

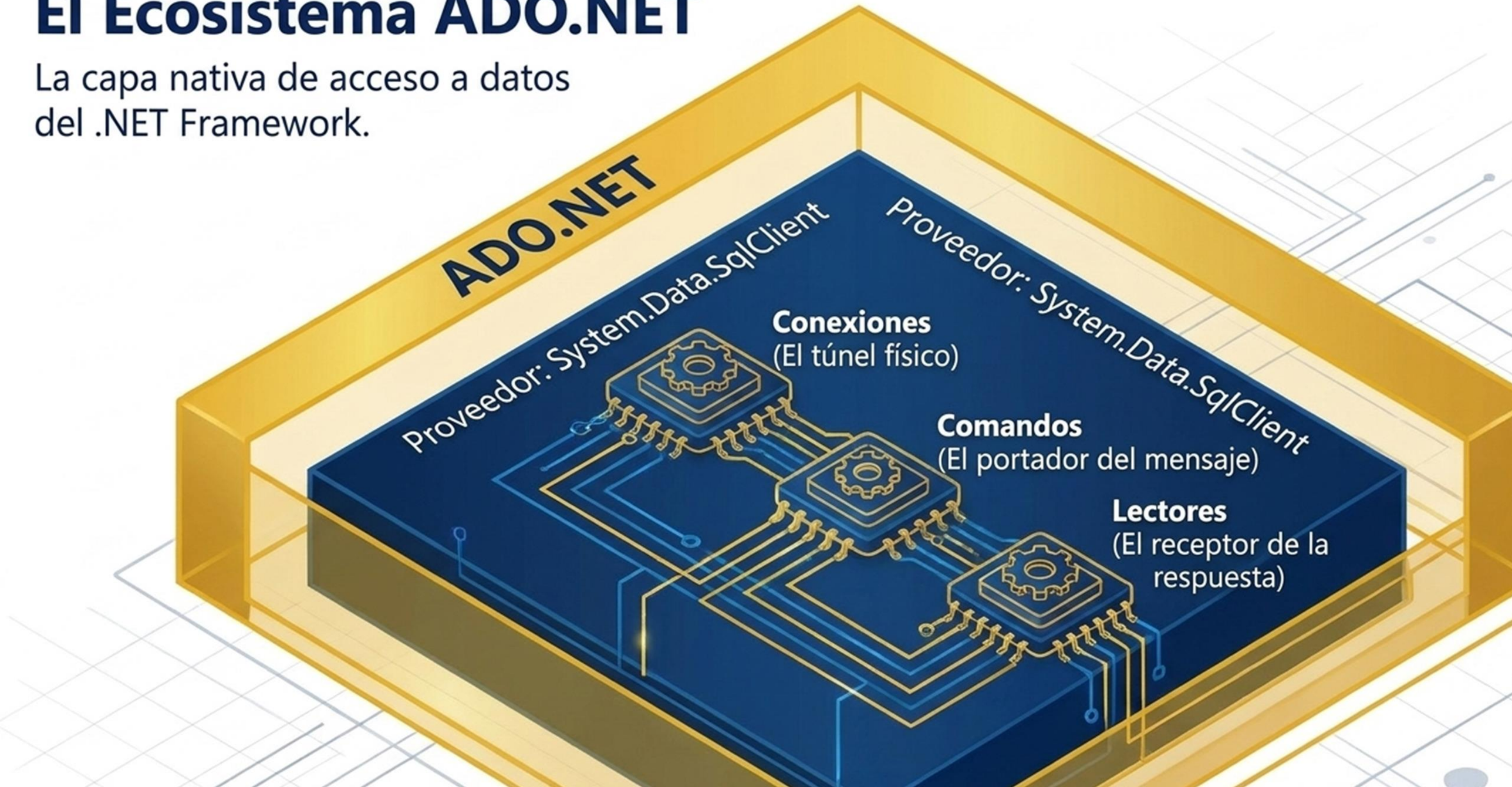
El motor local de desarrollo por excelencia: ligero, robusto y estándar en la industria .NET.



El **Problema de Comunicación:** C# no habla directamente con el disco duro para guardar tablas. Necesitamos construir un puente tecnológico intermediario que traduzca nuestras instrucciones al motor.

El Ecosistema ADO.NET

La capa nativa de acceso a datos del .NET Framework.



La Llave de Acceso: Cadenas de Conexión

Desglose técnico del ConnectionString

```
Server=.\SQLEXPRESS;
Database=PizzeriaDB;
Integrated Security=True;
```

1. Ubicación (Server)

¿Dónde está el motor? Apunta a la instancia local de la máquina.

2. Bóveda (Database)

¿A qué bóveda queremos entrar? El nombre exacto de nuestra base de datos.

3. Credenciales (Security)

¿Con qué permisos? Utiliza la autenticación segura de Windows del usuario actual.

Abriendo el Canal y Dando Órdenes

Paso 1: SqlConnection



- Representa el **túnel físico** entre la aplicación y la BD.
- **Requisito vital**: Necesita el ConnectionString.
- **Regla de Oro**: Se abre (Open) y se **cierra** (Close) inmediatamente al terminar.

Open()

Close()

Paso 2: SqlCommand



- Representa el **portador del mensaje**.
- Contiene la **instrucción** en lenguaje SQL puro (ej. INSERT INTO Pedidos...).
- Se ejecuta exclusivamente a través de una **conexión abierta**.

Sesión 2: Paradigmas de Acceso a Datos

Dos estrategias fundamentales para leer datos desde la base de datos.

Modelo Conectado



El túnel permanece abierto físicamente mientras se lee cada dato, fila por fila.

Modelo Desconectado



Se abre la conexión, se extrae una copia masiva de los datos hacia la memoria RAM, y se corta la comunicación física inmediatamente.

Matriz Comparativa: Elección del Modelo Adecuado

	Modelo Conectado	Modelo Desconectado
Objeto Principal	SqlDataReader	SqlDataAdapter + DataTable
Comportamiento	Flujo directo de solo avance. Conexión permanentemente activa.	Descarga una réplica completa a memoria. Conexión cerrada.
Ventaja Técnica	Altamente rápido. Muy bajo consumo de memoria RAM.	Permite navegar hacia atrás/adelante y vincular directamente a la UI.
Caso de Uso (WinForms)	Validar un Login o leer un dato único y específico.	Llenar un DataGridView interactivo con el menú o el historial de pedidos.

Solución Arquitectónica: Patrón DAO (Data Access Object)



Estructurando la Clase: PedidoDAO.cs

```
public class PedidoDAO
{
    // 1. Encapsula el ConnectionString privado de la base de datos
    private string connectionString = "Server=.\SQLEXPRESS; Database=PizzeriaDB...";

    // 2. Método de escritura (Implementa Modelo Conectado)
    public bool InsertarPedido(Pedido p)
    {
        // Lógica de SqlConnection y SqlCommand (INSERT)
    }

    // 3. Método de lectura masiva (Implementa Modelo Desconectado)
    public DataTable ObtenerPedidos()
    {
        // Lógica de SqlDataAdapter para llenar un DataTable (SELECT)
    }
}
```



Aislamiento Perfecto: El código de la interfaz WinForms queda completamente limpio, limitándose exclusivamente a llamar a estos métodos públicos.

El Flujo Desacoplado: Ciclo de Vida de una Operación

1



El usuario presiona 'Confirmar' en la interfaz.

WinForm captura los datos ingresados en pantalla y crea un objeto Pedido en memoria.

2



Delegación de Responsabilidad.

El WinForm llama al método PedidoDAO.InsertarPedido(nuevoPedido) y se queda esperando.

3



La Operación Técnica.

PedidoDAO abre la SqlConnection, ensambla el SqlCommand, lo ejecuta en la base de datos y cierra el túnel inmediatamente.

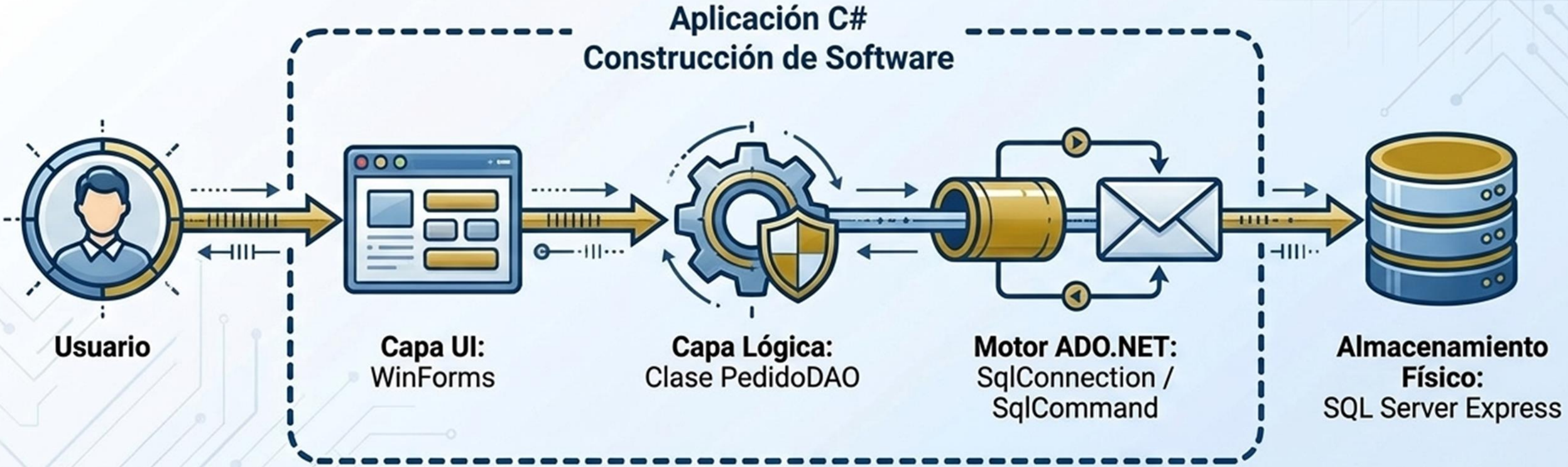
4



La Respuesta.

PedidoDAO retorna un estado de éxito (true). El WinForm recibe la respuesta y actualiza la pantalla (Ej: MessageBox.Show('Guardado con éxito')).

Síntesis Arquitectónica: El Cuadro Completo



Conclusión: Combinamos la eficiencia técnica de ADO.NET con el diseño estructural del Patrón DAO para construir aplicaciones mantenibles, escalables y blindadas.